

Infosys Interview Preparation Guide

Coding, Theoretical & Behavioral Questions with Answers

Compiled for: SP Off-Campus Drive Prep | Instagram: @make_it_worth19

How to Use This Document

Coding section : 40 questions collected from actual Infosys OA/HackWithInfy/interview reports (GeeksforGeeks, Glassdoor, PrepInsta, Naukri Code360) plus the standard SP/DSE topic bank. Solutions are in Python, with approach notes so you understand the pattern, not just memorize code.

Theoretical section : Core CS fundamentals (DBMS, OS, CN, OOP) as reported in real technical rounds.

Behavioral section : HR round questions with sample answer frameworks — adapt these with your own specifics (don't recite verbatim).

Infosys format reminder: **3 coding questions in 3 hours** (Easy/Medium/Hard), each with an independent sectional cutoff. You must clear every section — a strong Easy score does not offset a weak Hard score.

SECTION 1: CODING QUESTIONS (40)

A. Arrays & Strings

Q1. Two Sum *Given an array and a target, find indices of two numbers that add up to the target.*

Approach: Use a HashMap to store value→index while iterating once. For each element, check if **target - current** exists in the map.

```
python
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return [-1, -1]
```

Time: O(n), Space: O(n)

Q2. Reverse a String / Check Sentence Palindrome *Reverse a string, or check if a sentence reads the same after removing spaces/punctuation.*

Approach: Two-pointer technique from both ends, swapping characters. For sentence palindrome, first normalize (lowercase, strip non-alphanumeric), then apply two pointers.

```
python
```

```
import re

def is_palindrome(s):
    clean = re.sub(r"[^a-z0-9]", "", s.lower())
    left, right = 0, len(clean) - 1
    while left < right:
        if clean[left] != clean[right]:
            return False
        left += 1
        right -= 1
    return True
```

Time: $O(n)$, Space: $O(n)$

Q3. Longest Common Prefix Find the longest string prefix common to all strings in an array.

Approach: Take the first string as a candidate prefix. Repeatedly compare it against every other string, shrinking it until it matches, or return empty.

☐

```
python
def longest_common_prefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for i in range(1, len(strs)):
        while not strs[i].startswith(prefix):
            prefix = prefix[:-1]
        if not prefix:
            return ""
    return prefix
```

Time: $O(n*m)$, Space: $O(1)$

Q4. Valid Parentheses (reported: Glassdoor, May 2026 drive) Check if a string of brackets **()[]{}** is validly matched/nested.

Approach: Push opening brackets onto a stack. On a closing bracket, pop and check it matches the corresponding opener. String is valid if the stack is empty at the end.

☐

```
python
```

```
def is_valid(s):
    stack = []
    pairs = {'(': '(', ')': ')', '[': '[', ']': ']' }
    for c in s:
        if c in '([':
            stack.append(c)
        elif c in pairs:
            if not stack or stack.pop() != pairs[c]:
                return False
    return not stack
```

Time: O(n), Space: O(n)

Q5. Valid Anagram *Check if two strings are anagrams of each other.*

Approach: Count character frequencies of the first string in an array (size 26 for lowercase letters), then decrement using the second string. All counts must return to zero.

☐

```
python
def is_anagram(s, t):
    if len(s) != len(t):
        return False
    count = [0] * 26
    for c in s:
        count[ord(c) - ord('a')] += 1
    for c in t:
        count[ord(c) - ord('a')] -= 1
    return all(x == 0 for x in count)
```

Time: O(n), Space: O(1)

Q6. Move Zeroes *Move all zeroes in an array to the end while keeping relative order of non-zero elements, inplace.*

Approach: Two-pointer — one tracks the position to place the next non-zero element, the other scans the array.

☐

```
python
```

```
def move_zeroes(nums):
    insert_pos = 0
    for num in nums:
        if num != 0:
            nums[insert_pos] = num
            insert_pos += 1
    while insert_pos < len(nums):
        nums[insert_pos] = 0
        insert_pos += 1
```

Time: O(n), Space: O(1)

Q7. Kadane's Algorithm (Maximum Subarray Sum) *Find the contiguous subarray with the largest sum.*

Approach: Track the running sum ending at the current index; reset it to the current element if it drops below the element itself. Track the global maximum throughout.



```
python
def max_sub_array(nums):
    max_so_far = current_max = nums[0]
    for i in range(1, len(nums)):
        current_max = max(nums[i], current_max + nums[i])
        max_so_far = max(max_so_far, current_max)
    return max_so_far
```

Time: O(n), Space: O(1)

Q8. Subarray With a Given Sum / Count Subarrays Summing to K *Find (or count) subarrays whose sum equals a target k.*

Approach: Maintain a running prefix sum and a HashMap of prefix-sum frequencies. If **prefixSum - k** has been seen before, those earlier positions mark the start of valid subarrays.



```
python
def subarray_sum(nums, k):
    prefix_count = {0: 1}
    total = count = 0
    for num in nums:
        total += num
        count += prefix_count.get(total - k, 0)
        prefix_count[total] = prefix_count.get(total, 0) + 1
    return count
```

Time: $O(n)$, Space: $O(n)$

Q9. Merge Two Sorted Arrays ($O(1)$ extra space) Merge *arr2* into *arr1* in sorted order without extra space, given *arr1* has enough trailing capacity.

Approach: Fill from the back — compare the largest remaining elements of both arrays and place the bigger one at the end of *arr1*, moving pointers backward.

```
python
def merge(nums1, m, nums2, n):
    i, j, k = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i -= 1
        else:
            nums1[k] = nums2[j]
            j -= 1
        k -= 1
```

Time: $O(m+n)$, Space: $O(1)$

Q10. Rotate Array Rotate an array to the right by *k* steps, in-place.

Approach: Reverse the whole array, then reverse the first *k* elements, then reverse the remaining *n-k* elements. This gives the rotated result in three linear passes.

```
python
def rotate(nums, k):
    n = len(nums)
    k %= n

    def reverse(l, r):
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1
            r -= 1

    reverse(0, n - 1)
    reverse(0, k - 1)
    reverse(k, n - 1)
```

Time: $O(n)$, Space: $O(1)$

Q11. Container With Most Water *Given heights, find two lines that with the x-axis form the container holding the most water.*

Approach: Two pointers starting at both ends. Move the pointer at the shorter line inward each step, since moving the taller one can never increase the area.

```
python
def max_area(height):
    left, right, best = 0, len(height) - 1, 0
    while left < right:
        area = min(height[left], height[right]) * (right - left)
        best = max(best, area)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

Time: $O(n)$, Space: $O(1)$

Q12. Product of Array Except Self *Return an array where each element is the product of all others (no division allowed).*

Approach: Two passes — build prefix products from the left, then multiply in suffix products from the right in a single combined array.

```
python
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    for i in range(1, n):
        result[i] = result[i - 1] * nums[i - 1]
    right = 1
    for i in range(n - 1, -1, -1):
        result[i] *= right
        right *= nums[i]
    return result
```

Time: $O(n)$, Space: $O(1)$ excluding output

Q13. Search in Rotated Sorted Array *Binary search on an array that was sorted then rotated at an unknown pivot.*

Approach: Modified binary search — at each step, determine which half (left or right of mid) is properly sorted, then check if target lies within that sorted half to decide direction.



python

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Time: $O(\log n)$, Space: $O(1)$

Q14. String to Integer (atoi) *Convert a string to an integer, handling whitespace, sign, overflow, and invalid characters.*

Approach: Skip leading whitespace, capture an optional sign, then consume digits while accumulating the result, clamping to INT_MAX/INT_MIN on overflow.



python

```

def my_atoi(s):
    i, n = 0, len(s)
    while i < n and s[i] == ' ':
        i += 1
    if i == n:
        return 0
    sign = 1
    if s[i] in '+-':
        sign = -1 if s[i] == '-' else 1
        i += 1
    result = 0
    INT_MAX, INT_MIN = 2**31 - 1, -2**31
    while i < n and s[i].isdigit():
        result = result * 10 + int(s[i])
        i += 1
    if result * sign >= INT_MAX:
        return INT_MAX
    if result * sign <= INT_MIN:
        return INT_MIN
    return result * sign

```

Time: O(n), Space: O(1)

Q15. Longest Palindromic Substring *Find the longest substring that is a palindrome.*

Approach: Expand-around-center — for every index (and every gap between indices, for even-length palindromes), expand outward while characters match, tracking the longest found.



python

```

def longest_palindrome(s):
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1
            r += 1
        return r - l - 1

    start, max_len = 0, 0
    for i in range(len(s)):
        len1 = expand(i, i)
        len2 = expand(i, i + 1)
        length = max(len1, len2)
        if length > max_len:
            max_len = length
            start = i - (length - 1) // 2
    return s[start:start + max_len]

```

Time: $O(n^2)$, Space: $O(1)$

Q16. Minimum Window Substring Find the smallest substring of *s* containing all characters of *t* (with frequency).

Approach: Sliding window with two pointers. Expand the right pointer until the window contains all needed characters, then contract the left pointer to shrink the window while it stays valid, tracking the minimum.



python

```

from collections import Counter

def min_window(s, t):
    need = Counter(t)
    window = {}
    have, need_count = 0, len(need)
    left, res_len, res_start = 0, float('inf'), 0
    for right, c in enumerate(s):
        window[c] = window.get(c, 0) + 1
        if c in need and window[c] == need[c]:
            have += 1
        while have == need_count:
            if right - left + 1 < res_len:
                res_len = right - left + 1
                res_start = left
            lc = s[left]
            window[lc] -= 1
            if lc in need and window[lc] < need[lc]:
                have -= 1
            left += 1
    return "" if res_len == float('inf') else s[res_start:res_start + res_len]

```

Time: O(n), Space: O(k)

Q17. Maximum Sum Subarray with At Most K Distinct Elements (reported: HackWithInfy pattern bank)

Find max sum of a contiguous subarray containing at most K distinct elements.

Approach: Sliding window + HashMap tracking element frequencies in the window. Expand right; whenever distinct count exceeds K, shrink from the left until valid again. Track running sum and max.



python

```
def max_sum_k_distinct(arr, k):
    freq = {}
    left = total = max_sum = 0
    for right in range(len(arr)):
        total += arr[right]
        freq[arr[right]] = freq.get(arr[right], 0) + 1
        while len(freq) > k:
            total -= arr[left]
            freq[arr[left]] -= 1
            if freq[arr[left]] == 0:
                del freq[arr[left]]
            left += 1
        max_sum = max(max_sum, total)
    return max_sum
```

Time: O(n), Space: O(k)

Q18. Swap Two Numbers Without a Third Variable (*reported: basic warm-up question*)

Approach: Use arithmetic (or XOR) to swap without a temp variable.

☐

```
python
def swap(a, b):
    a = a + b
    b = a - b
    a = a - b
    print(f"a={a} b={b}")
```

Time: O(1), Space: O(1)

Q19. Fahrenheit to Celsius Conversion (*reported: basic I/O formatting test*) Tests exact output formatting — Infosys grading is strict on spacing/newlines.

☐

```
python
def fahrenheit_to_celsius(f):
    return (5.0 / 9.0) * (f - 32)
```

Note: Always double-check exact expected output format (decimal places, trailing text) — this trips up more candidates than the logic itself.

Q20. Maximum XOR of Two Arrays (*reported: SP off-campus round, hard tier*) Find max value of **a XOR b** where a is from arr1 and b is from arr2.

Approach (brute force, acceptable for smaller constraints): iterate all pairs. For larger constraints, use a bitwise Trie built from one array, then query greedily bit-by-bit for each element of the other array to maximize XOR.

```
python
# Brute force O(n*m) - fine if constraints are small
def find_max_xor(arr1, arr2):
    return max(a ^ b for a in arr1 for b in arr2)
```

For large inputs, mention the Trie-based $O(n \log(\max_val))$ approach in your interview even if you code the brute force first — shows you know the optimization. ### B. Backtracking (your focus area)

Q21. Generate Parentheses *Generate all combinations of n pairs of well-formed parentheses.*

Approach: Backtrack, tracking counts of open and close brackets used. Add an open bracket if you haven't used n yet; add a close bracket only if close count < open count.

```
python
def generate_parenthesis(n):
    result = []

    def backtrack(sb, open_count, close_count):
        if len(sb) == n * 2:
            result.append("".join(sb))
            return
        if open_count < n:
            sb.append('(')
            backtrack(sb, open_count + 1, close_count)
            sb.pop()
        if close_count < open_count:
            sb.append(')')
            backtrack(sb, open_count, close_count + 1)
            sb.pop()

    backtrack([], 0, 0)
    return result
```

Time: $O(4^n / \sqrt{n})$, Space: $O(n)$ recursion depth

Q22. N-Queens *Place N queens on an $N \times N$ board so no two attack each other.*

Approach: Place queens row by row. For each row, try each column; check if it's safe (no queen in same column or diagonal) using tracking sets, then recurse to the next row.

```
python
def solve_n_queens(n):
    result = []
    queens = [0] * n
    cols, diag1, diag2 = set(), set(), set()

    def build_board():
        board = []
        for q in queens:
            row = ['.'] * n
            row[q] = 'Q'
            board.append(''.join(row))
        return board

    def solve(row):
        if row == n:
            result.append(build_board())
            return
        for col in range(n):
            d1, d2 = row - col, row + col
            if col in cols or d1 in diag1 or d2 in diag2:
                continue
            queens[row] = col
            cols.add(col)
            diag1.add(d1)
            diag2.add(d2)
            solve(row + 1)
            cols.remove(col)
            diag1.remove(d1)
            diag2.remove(d2)

    solve(0)
    return result
```

Time: $O(N!)$, Space: $O(N)$

Q23. Palindrome Partitioning *Partition a string such that every substring in the partition is a palindrome.*

Approach: Backtrack over starting index. Try every possible end point for the current substring; if it's a palindrome, recurse on the remaining string.

```
python
def partition(s):
    result = []

    def is_pal(sub):
        return sub == sub[::-1]

    def backtrack(start, path):
        if start == len(s):
            result.append(list(path))
            return
        for end in range(start + 1, len(s) + 1):
            sub = s[start:end]
            if is_pal(sub):
                path.append(sub)
                backtrack(end, path)
                path.pop()

    backtrack(0, [])
    return result
```

Time: $O(n * 2^n)$, Space: $O(n)$

Q24. Word Ladder (conceptually close to a reported HackWithInfy question) Transform *startWord* to *endWord*, changing one letter at a time, each intermediate word must be in the word list. Find shortest transformation length.

Approach: BFS from *startWord*. At each level, try changing every character position to every other letter and check if the result is in the word set — this is a shortest-path problem on an implicit graph, so BFS (not DFS/backtracking) gives the minimum steps.

```

from collections import deque

def ladder_length(begin_word, end_word, word_list):
    word_set = set(word_list)
    if end_word not in word_set:
        return 0
    queue = deque([begin_word])
    steps = 1
    while queue:
        for _ in range(len(queue)):
            word = list(queue.popleft())
            for j in range(len(word)):
                original = word[j]
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    word[j] = c
                    next_word = ''.join(word)
                    if next_word == end_word:
                        return steps + 1
                    if next_word in word_set:
                        word_set.remove(next_word)
                        queue.append(next_word)
                word[j] = original
            steps += 1
    return 0

```

Time: $O(M^2 * N)$ where M = word length, N = word list size

C. Trees & Graphs

Q25. Binary Tree Traversals (In-order, Pre-order, Post-order)



python

```
def inorder(root, res):
    if root is None:
        return
    inorder(root.left, res)
    res.append(root.val)
    inorder(root.right, res)

def preorder(root, res):
    if root is None:
        return
    res.append(root.val)
    preorder(root.left, res)
    preorder(root.right, res)

def postorder(root, res):
    if root is None:
        return
    postorder(root.left, res)
    postorder(root.right, res)
    res.append(root.val)
```

Time: $O(n)$, Space: $O(h)$ where h = tree height

Q26. Lowest Common Ancestor (LCA) of a Binary Tree

Approach: Recurse into both subtrees. If both left and right recursion return non-null, the current node is the LCA. Otherwise propagate whichever side found a match.



```
python
def lowest_common_ancestor(root, p, q):
    if root is None or root == p or root == q:
        return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right:
        return root
    return left if left else right
```

Time: $O(n)$, Space: $O(h)$

Q27. Validate Binary Search Tree

Approach: Recurse while carrying a valid (min, max) range for each node. Each node's value must fall strictly within that range, and the range narrows as you descend.



python

```
def is_valid_bst(root):
    def validate(node, lower, upper):
        if node is None:
            return True
        if lower is not None and node.val <= lower:
            return False
        if upper is not None and node.val >= upper:
            return False
        return validate(node.left, lower, node.val) and validate(node.right, node.val, upper)

    return validate(root, None, None)
```

Time: O(n), Space: O(h)

Q28. BFS Traversal (Graph)



python

```
from collections import deque

def bfs(start, adj):
    result = []
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in adj.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return result
```

Time: O(V+E), Space: O(V)

Q29. DFS Traversal (Graph)



python

```
def dfs(node, adj, visited, result):
    visited.add(node)
    result.append(node)
    for neighbor in adj.get(node, []):
        if neighbor not in visited:
            dfs(neighbor, adj, visited, result)
```

Time: $O(V+E)$, Space: $O(V)$

Q30. Detect Cycle in a Graph (Directed & Undirected)

Approach (directed): DFS with a recursion stack — if you revisit a node that's currently in the recursion stack, there's a cycle. Approach (undirected): DFS/Union-Find — if you visit a neighbor that's already visited and isn't the immediate parent, there's a cycle.

☐

```
python
# Directed graph cycle detection
def has_cycle_directed(node, adj, visited, rec_stack):
    visited.add(node)
    rec_stack.add(node)
    for neighbor in adj.get(node, []):
        if neighbor in rec_stack:
            return True
        if neighbor not in visited and has_cycle_directed(neighbor, adj, visited, rec_stack):
            return True
    rec_stack.remove(node)
    return False
```

Time: $O(V+E)$, Space: $O(V)$

Q31. Dijkstra's Algorithm *(reported: asked to explain with pseudocode in a technical round)* Find shortest paths from a source node to all other nodes in a weighted graph (non-negative weights).

Approach: Use a min-priority-queue. Start with distance 0 at source, infinity elsewhere. Repeatedly pop the closest unvisited node, relax its edges (update neighbor distances if a shorter path is found via this node), and push updated distances back.

☐

```
python
```

```

import heapq

def dijkstra(n, adj, src):
    dist = [float('inf')] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, weight in adj[u]:
            if dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight
                heapq.heappush(pq, (dist[v], v))
    return dist

```

Time: $O((V+E) \log V)$, Space: $O(V)$

Q32. Topological Sort

Approach: Kahn's algorithm (BFS-based) — compute in-degree for every node, start a queue with all zero-in-degree nodes, repeatedly pop and reduce neighbors' in-degrees, pushing any that reach zero.

☐

```

python
from collections import deque

def topo_sort(n, edges):
    adj = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        adj[u].append(v)
        indegree[v] += 1
    queue = deque(i for i in range(n) if indegree[i] == 0)
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for nxt in adj[node]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    return result

```

Time: $O(V+E)$, Space: $O(V)$

D. Dynamic Programming

Q33. 0/1 Knapsack Maximize value in a knapsack of capacity W , each item used at most once.

Approach: 2D DP where $dp[i][w]$ = max value using first i items with capacity w . For each item, choose to include it (if it fits) or skip it, taking the better option.

```
python
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(W + 1):
            dp[i][w] = dp[i - 1][w]
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i][w], dp[i - 1][w - weights[i - 1]] + values[i - 1])
    return dp[n][W]
```

Time: $O(n W)$, Space: $O(n W)$

Q34. Longest Common Subsequence

Approach: 2D DP where $dp[i][j]$ = LCS length of first i chars of $s1$ and first j chars of $s2$. If characters match, extend diagonally; otherwise take the max of skipping one character from either string.

```
python
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Time: $O(m n)$, Space: $O(m n)$

Q35. Maximum Product Subarray (reported: SP off-campus round, medium tier) Find contiguous subarray with the largest product.

Approach: Track both a running max and running min ending at each index (a negative number can flip a min into a max). At each step, update both using the current element.



python

```
def max_product(nums):
    max_prod = min_prod = result = nums[0]
    for i in range(1, len(nums)):
        curr = nums[i]
        if curr < 0:
            max_prod, min_prod = min_prod, max_prod
        max_prod = max(curr, max_prod * curr)
        min_prod = min(curr, min_prod * curr)
        result = max(result, max_prod)
    return result
```

Time: $O(n)$, Space: $O(1)$

Q36. Climbing Stairs *Count distinct ways to climb n stairs, taking 1 or 2 steps at a time.*

Approach: Classic Fibonacci-style DP — ways to reach step n = ways to reach $(n-1)$ + ways to reach $(n-2)$.



python

```
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for i in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

Time: $O(n)$, Space: $O(1)$

Q37. House Robber *Maximize sum of non-adjacent elements picked from an array.*

Approach: DP tracking two states — max amount if you rob up to the previous house, and up to the one before that. At each house, choose max of (skip this house) or (rob this house + best from two houses back).



python

```
def rob(nums):
    prev2 = prev1 = 0
    for num in nums:
        curr = max(prev1, prev2 + num)
        prev2, prev1 = prev1, curr
    return prev1
```

Time: $O(n)$, Space: $O(1)$

Q38. LCM of Array + Minimum Subsequence with Same LCM (reported: HackWithInfy panel round — closely paraphrased) Compute the LCM of an entire array = L . Find the minimum-length subsequence whose LCM also equals L .

Approach: First compute overall LCM L . Then greedily check which individual elements' prime-factor contributions are actually needed to reach L — an element is “essential” if removing it would lower the LCM. Practically: for each prime factor power present in L , at least one array element must supply that exact power; find the minimal set covering all such factor requirements.

☐

```
python
def gcd(a, b):
    return a if b == 0 else gcd(b, a % b)

def lcm(a, b):
    return a // gcd(a, b) * b

def array_lcm(arr):
    result = arr[0]
    for i in range(1, len(arr)):
        result = lcm(result, arr[i])
    return result
```

```
# For the minimal subsequence: factorize L, then greedily pick elements
# that uniquely supply each prime power in L's factorization (interview
# discussion question — explain the factor-coverage approach verbally
# even if full code isn't required).
```

Time: $O(n \log(\max))$ for LCM, factorization approach is $O(\sqrt{L})$ additional

Q39. Explain Time/Space Complexity of Any Solution (asked after nearly every coding question)

This isn't a single “question” — it's a standing expectation. For every solution above, be ready to state: - Time complexity in Big-O, and *why* (e.g., “single pass with $O(1)$ HashMap lookups = $O(n)$ ”) - Space complexity,

distinguishing auxiliary space from output space - Whether an approach can be optimized further (e.g., “this could be reduced from $O(n^2)$ to $O(n)$ using a HashMap”)

Q40. Basic OOP-in-Code (write a small class hierarchy live) (reported: technical round expectation)
Interviewers may ask you to define a simple class hierarchy on the spot, not just recite OOP definitions.

```
python
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width
```

This demonstrates abstraction, inheritance, and polymorphism (**area()** overridden differently per subclass) in one compact example — a good go-to if asked to “show OOP in code.” ## SECTION 2: THEORETICAL / CS FUNDAMENTALS

DBMS & SQL

Q1. Explain SQL vs NoSQL with examples. SQL databases (MySQL, PostgreSQL, SQL Server) are relational, use structured schemas with tables and fixed columns, and enforce ACID properties strongly — good for transactional, structured data like banking records. NoSQL databases (MongoDB, Cassandra, Redis) are non-relational, support flexible/schema-less structures (documents, key-value, wide-column, graph), and prioritize horizontal scalability — good for unstructured or rapidly evolving data like user activity logs or session data.

Q2. Explain BCNF (Boyce-Codd Normal Form). BCNF is a stricter version of 3NF. A relation is in BCNF if, for every functional dependency $X \rightarrow Y$, X must be a super key. It resolves anomalies that 3NF sometimes misses, specifically when a table has multiple overlapping candidate keys with dependencies between them.

Q3. What are the different normalization forms (1NF-3NF)? - 1NF: Eliminate repeating groups — each column holds atomic values. - 2NF: 1NF + no partial dependency (non-key attributes depend on the whole primary key, not part of it). - 3NF: 2NF + no transitive dependency (non-key attributes depend only on the primary key, not on other non-key attributes).

Q4. Explain the ACID properties. Atomicity (transaction is all-or-nothing), Consistency (database moves between valid states), Isolation (concurrent transactions don't interfere), Durability (committed changes persist even after a crash).

Q5. Difference between joins (INNER, LEFT, RIGHT, FULL). INNER JOIN returns only matching rows in both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULLs where no match). RIGHT JOIN is the mirror of LEFT. FULL JOIN returns all rows from both tables, matched where possible.

Q6. Why use indexing, and what's the tradeoff? Indexes speed up read/query performance by allowing the database to avoid full table scans (typically via B-Trees). The tradeoff is slower writes (inserts/updates/deletes) since indexes must be updated too, plus extra storage overhead.

Operating Systems

Q7. Explain the Banker's Algorithm. A deadlock-avoidance algorithm. Before granting a resource request, the OS simulates whether granting it would leave the system in a "safe state" (i.e., all processes can still eventually complete). If the resulting state is unsafe, the request is denied/delayed.

Q8. Explain demand paging. Pages are loaded into physical memory only when they're actually accessed (referenced), not all at once at process start. This reduces memory waste and startup time, at the cost of page faults when a needed page isn't yet loaded.

Q9. Difference between Monolithic Kernel and Microkernel. Monolithic kernel runs all OS services (file system, drivers, memory management) in one large kernel space — fast but less modular/stable (a bug in one component can crash the whole kernel). Microkernel keeps only essential services (IPC, basic scheduling) in kernel space and runs others (drivers, file systems) as user-space processes — more stable/modular but slower due to inter-process communication overhead.

Q10. Process vs Thread. A process is an independent execution unit with its own memory space. A thread is a lightweight unit within a process, sharing the process's memory but having its own stack and register state. Threads are cheaper to create/switch between than processes.

Q11. What are the necessary conditions for deadlock? Mutual exclusion, hold and wait, no preemption, and circular wait — all four must hold simultaneously for a deadlock to occur.

Computer Networks

Q12. Difference between IPv4 and IPv6. IPv4 uses 32-bit addresses (~4.3 billion addresses, dotted-decimal notation like 192.168.1.1). IPv6 uses 128-bit addresses (vastly larger address space, hexadecimal notation), was

introduced to solve IPv4 exhaustion, and includes built-in improvements like simplified header format and better native security support.

Q13. TCP vs UDP. TCP is connection-oriented, reliable (guarantees delivery and order via acknowledgments/retransmission), but slower due to overhead — used for web browsing, file transfer, email. UDP is connectionless, faster, no delivery guarantee — used for video streaming, gaming, DNS lookups where speed matters more than perfect reliability.

Q14. Explain the OSI model layers briefly. Physical (raw bits) → Data Link (frames, MAC addressing) → Network (routing, IP) → Transport (TCP/UDP, end-to-end delivery) → Session (managing connections) → Presentation (data formatting/encryption) → Application (user-facing protocols like HTTP).

OOP

Q15. Explain the four pillars of OOP with examples. - Encapsulation: bundling data and methods together, restricting direct access (e.g., private fields with public getters/setters). - Abstraction: exposing only essential features, hiding implementation (e.g., an abstract **Shape** class with an **area()** method, hiding how each shape calculates it). - Inheritance: a subclass acquires properties/behavior of a parent class (e.g., **Rectangle extends Shape**). - Polymorphism: same method behaves differently based on the object calling it — method overriding (runtime) or overloading (compile-time).

Q16. Difference between method overloading and overriding. Overloading: same method name, different parameter lists, within the same class — resolved at compile time. Overriding: subclass redefines a parent class method with the same signature — resolved at runtime (dynamic dispatch), enables polymorphism.

Q17. What is a constructor, and can it be overloaded? A constructor initializes an object when it's created, sharing the class name and no return type. Yes — a class can have multiple constructors with different parameter lists (constructor overloading).

Q18. Explain object-oriented programming's advantage over procedural programming. OOP models realworld entities as objects combining data and behavior, improving modularity, reusability (via inheritance), and maintainability, versus procedural programming's linear sequence of function calls operating on shared data.

Algorithms (explain-on-the-spot questions)

Q19. Explain how binary search works. Repeatedly divide a sorted array's search space in half — compare the target to the middle element; if equal, done; if target is smaller, search the left half; if larger, search the right half. Continues until found or the search space is empty. $O(\log n)$.

Q20. Explain Dijkstra's Algorithm with pseudocode. See Q31 in the coding section above for full code — verbally, describe it as: greedy shortest-path algorithm using a priority queue, always expanding the currently closest unvisited node and relaxing its edges, works only with non-negative edge weights.

Q21. What's the difference between BFS and DFS, and when would you use each? BFS explores level-by-level using a queue — best for shortest path in unweighted graphs. DFS explores as deep as possible before backtracking, using a stack (or recursion) — best for cycle detection, topological sort, and exhaustive search (like backtracking problems). ## SECTION 3: BEHAVIORAL / HR QUESTIONS

Note: These are frameworks, not scripts. Fill in your own specifics — interviewers can tell when an answer is memorized word-for-word versus genuinely yours.

Q1. Tell me about yourself. Structure: (1) Who you are + degree, (2) technical focus/strength, (3) a standout project or achievement, (4) what you're looking for now. *Framework:* "I'm a recent B.Tech ECE graduate from IARE, Hyderabad, but my focus through college shifted toward software development — specifically full-stack and .NET. I've built and deployed production projects like an AI-powered interview prep platform using .NET 9, React, and

Azure AI services, and I've solved 300+ DSA problems on LeetCode. I'm now looking for a role where I can apply that hands-on development experience on real-world problems."

Q2. Why do you want to join Infosys? Anchor in something specific — Infosys's scale, training program (ILP), or the SP/DSE track itself — not generic flattery. *Framework:* "Infosys gives freshers a structured path to work on large-scale, real client projects with strong foundational training. The SP/DSE track specifically appeals to me because it's coding-and-DSA-focused, which matches how I've spent the last year preparing."

Q3. Where do you see yourself in 5 years? Show ambition balanced with realism and loyalty signal — don't say you want to job-hop. *Framework:* "I want to grow from being a strong individual contributor into someone who can lead technical decisions on a project — ideally moving toward architecture or technical leadership, while continuing to build deep expertise in full-stack development."

Q4. Describe a challenge you overcame. Use STAR: Situation, Task, Action, Result. Pull from a real project — e.g., the 24-hour deadline on your Real-Time Chat Application, or a bug you had to debug under pressure. *Framework:* "While building my real-time chat app, I had a hard 24-hour deadline. Midway through, Socket.io events weren't syncing correctly across clients. I isolated the issue to a race condition in how I was broadcasting events, restructured the event handling, and shipped on time with working real-time messaging."

Q5. Describe a mistake you learned from. Pick something real but not catastrophic — show self-awareness and a concrete fix, not just "I worked too hard." *Framework:* "Early in a project, I didn't plan my database schema carefully enough, which caused rework later when I needed to add a feature. I've since made schema planning and EF Core migrations something I do upfront, before writing feature code."

Q6. How do you handle conflict or pressure in a team? Show calm, structured problem-solving — not conflict avoidance, not aggression. *Framework:* "I try to separate the disagreement from the person — focus on what the project actually needs, lay out the tradeoffs of each approach, and if we're stuck, defer to data (tests, benchmarks) rather than opinion."

Q7. What are your strengths and weaknesses? Strength: pick something backed by evidence (e.g., "I'm consistent — I solved 300+ LeetCode problems methodically over months"). Weakness: pick something real but already being actively worked on — not a disguised strength like "I work too hard." *Framework weakness:* "I used to underestimate how long polishing/testing takes versus writing the core logic. I've started blocking dedicated time for testing and edge cases in every project now."

Q9. Walk me through your project (expect deep follow-ups). Be ready to go 3-4 levels deep on any project on your resume — architecture, specific technical decisions, why you chose one tool over another.

Framework starting point: “My Interview Prep Platform is a full-stack app — .NET 9 Web API backend with six EF Core entities, JWT authentication, and PostgreSQL for storage. On the AI side, I integrated Groq’s Llama 3.3 70B model via API for generating practice questions, and used Azure AI Search for retrieval. The frontend is React with Recharts for progress dashboards.” Then be ready for: “Why Groq over OpenAI?”, “How did you handle auth token expiry?”, “What was the hardest part?”

Q10. What do you know about Infosys? Know the basics cold: founded by N.R. Narayana Murthy and six cofounders in 1981; headquartered in Bengaluru; one of India’s largest IT services and consulting companies; major service lines include cloud, AI, digital transformation, cybersecurity, and consulting; runs InfyTQ and HackWithInfy as fresher hiring/certification pipelines.

Q11. Are you fine with a probation period or bond conditions (if applicable to your offer)? Answer honestly, but don’t create friction here unless you have a genuine, serious concern — ask for the specific terms in writing if unsure rather than debating it live in the interview.

Q12. A time you worked in a team / took a leadership role. Pull from any group project, internship (Zeotap), or even a personal project where you coordinated with someone else (e.g., building the birthday website for a friend involved understanding someone else’s requirements/vision).

Q13. Any questions for us? (*always asked — always have 2-3 ready*) Good options: “What does the onboarding and training path look like for someone joining as SP?” / “What kind of projects does the DSE track typically get staffed on?” / “What does success look like in the first 6 months for someone in this role?” Avoid asking purely about salary/perks at this stage — save that for the HR/offer discussion.

Quick Reference: Format Summary

Round	Format	Duration	Notes
OA / IRT	~64 objective Qs + essay	120-180 min	Independent sectional cutoffs
HackWithInfy R1	3 coding Qs (easy→medium→hard)	3 hours	Array / String / Graph pattern reported
HackWithInfy R2 (Advantage)	3 coding Qs, medium-hard	~3 hours	DP, Segment Trees, Graphs, Sliding Window
Pre-interview coding filter	2 coding Qs, medium	Varies	DP most common topic
Technical Interview	Live coding + CS fundamentals + project deep-dive	30-45 min	Explain time/space complexity always
HR / Behavioral	STAR-based, culture fit	15-30 min	Confidence and communication matter as much as content

Final tip from real candidate reports: interviewers may observe your problem-solving process directly (checking rough sheets, watching you think) — narrate your approach out loud as you solve, don't just code silently.

DSA Problem Set

Problem statements, LeetCode references, approach, and verified Python solutions

Streetlamps With Generator Truck

Combination Sum IV (Alternating Parity) — based on LeetCode 377

Best Time to Buy and Sell Stock II (Fixed Holding Period) — related to LeetCode 122

Stock Trading with Settlement Delay — related to LeetCode 121

Maximum Value of K Coins From Piles — based on LeetCode 2218

Coin Change II (Minimum Denomination Difference) — based on LeetCode 518

Edit Distance — LeetCode 72

Longest Increasing Path in a Matrix — LeetCode 329

Frog Jump with Minimum Fatigue — Custom Problem

Question 1: Streetlamps With Generator Truck

LeetCode: No exact match. Thematically related to LeetCode 1326 (Minimum Number of Taps to Open to Water a Garden) — an interval-covering problem — but this variant restricts most device placements while

allowing one free, unrestricted-placement device.

Problem Statement

A dead-end service road has N posts in a line, numbered 1 to N . A streetlamp installed on post i lights post i and its immediate neighbors $i-1$ and $i+1$ (if they exist). Every post must be lit.

There are two utility conditions:

Dead posts: Some posts are marked dead, represented by a string z of length N consisting of characters 'L' (live) and 'D' (dead). A dead post cannot hold a lamp, but it must still be lit by a neighbor's lamp or the generator truck.

The generator truck: You have at most one generator truck that can be parked at any one post of your choice (live or dead). The truck floodlights its post and both immediate neighbors (just like a lamp) but does not count towards the lamp cost.

Find the minimum number of lamps required to light all N posts, or return -1 if it is impossible to light all posts even with the generator truck.

Input Format

Line 1: integer N . Line 2: string z of length N ('L'/'D' status of each post).

Constraints

$$1 \leq N \leq 10^5$$

Examples

$N=1, z="L" \rightarrow$ Output: 0. $N=6, z="LLDDL" \rightarrow$ Output: 1. $N=9, z="LLLDDDLL" \rightarrow$ Output: 2.

Approach

Process the leftmost still-unlit post p left to right. At each such post, greedily pick the *rightmost* valid device center among $\{p-1, p, p+1\}$ — covering further right is never worse. Two options: a lamp (valid only if some position in that window is 'L'), or, if the truck hasn't been used yet, the truck (always available, unrestricted). This gives a simple DP with 2 states per position (truck available or not), solved in $O(N)$.

Python Solution



python

```

import sys

def solve(N, z):
    INF = float('inf')
    f = [[0, 0] for _ in range(N + 2)]
    for p in range(N, 0, -1):
        lamp_center = None
        for c in (p + 1, p, p - 1):
            if 1 <= c <= N and z[c - 1] == 'L':
                lamp_center = c
                break
        truck_center = p + 1 if p + 1 <= N else p
        for t in range(2):
            best = INF
            if lamp_center is not None:
                nxt = min(lamp_center + 2, N + 1)
                best = min(best, 1 + f[nxt][t])
            if t == 1:
                nxt = min(truck_center + 2, N + 1)
                best = min(best, f[nxt][0])
            f[p][t] = best
    answer = f[1][1]
    return -1 if answer >= INF else answer

def main():
    data = sys.stdin.read().split()
    N = int(data[0])
    z = data[1]
    print(solve(N, z))

if __name__ == "__main__":
    main()

```

Complexity: $O(N)$ time, $O(N)$ space. Verified against brute force over 400 random cases plus all samples.

Question 2: Combination Sum IV (Alternating Parity)

LeetCode: Based on LeetCode 377 (Combination Sum IV), with an added constraint that no two adjacent elements in the ordered combination may share the same parity.

Problem Statement

You are given an array of distinct integers `nums` and an integer `target`. Return the number of ordered combinations (sequences) of `nums` that sum to `target` such that no two adjacent elements have the same

parity (they must strictly alternate between even and odd). Different orderings count separately. Return the count modulo 10^9+7 .

Input Format

Line 1: integer N. Line 2: N space-separated distinct integers, `nums[i]`. Line 3: integer target.

Constraints

$1 \leq N \leq 200$ | $1 \leq \text{nums}[i] \leq 1000$ | $1 \leq \text{target} \leq 1000$

Examples

`nums=[1,2,3]`, `target=4` → Output: 1 (only (1,2,1)). `nums=[2,4,6]`, `target=8` → Output: 0. `nums=[1,2]`, `target=3` → Output: 2.

Approach

Standard Combination Sum IV DP extended with a parity dimension: `dp[s][0]/dp[s][1]` = number of valid alternating sequences summing to `s` that END with an even/odd number. A "start" value of 1 represents the empty sequence. For each sum `s` and each `num`, if it's the first element of the sequence, add to `dp[s][parity(num)]`; otherwise it must follow the OPPOSITE parity, so add `dp[s-num][1-parity(num)]`. Answer = `dp[target][0] + dp[target][1]`.

Python Solution



python

```

import sys

def solve(N, nums, target):
    MOD = 1000000007
    dp = [[0, 0] for _ in range(target + 1)]
    start = 1
    for s in range(1, target + 1):
        for num in nums:
            if num > s:
                continue
            par = num % 2
            if s - num == 0:
                dp[s][par] = (dp[s][par] + start) % MOD
            else:
                dp[s][par] = (dp[s][par] + dp[s - num][1 - par]) % MOD
    return (dp[target][0] + dp[target][1]) % MOD

def main():
    data = sys.stdin.read().split()
    idx = 0
    N = int(data[idx]); idx += 1
    nums = [int(data[idx + i]) for i in range(N)]; idx += N
    target = int(data[idx])
    print(solve(N, nums, target))

if __name__ == "__main__":
    main()

```

Complexity: $O(N \cdot \text{target})$ time, $O(\text{target})$ space. Verified against brute force over 300 random cases plus all samples.

Question 3: Best Time to Buy and Sell Stock II

LeetCode: Thematically related to LeetCode 122 (Best Time to Buy and Sell Stock II), but with a hard constraint: a sale must happen exactly K days after purchase, not just any later day.

Problem Statement

You are given an integer array `prices`, where `prices[i]` is the price of a stock on day i , and an integer K . You may perform as many transactions as you like, subject to:

If you buy on day i , you must sell it exactly K days later, on day $i + K$.

You may hold at most one share at any time.

After selling on day $i + K$, you may immediately buy another share the same day.

Determine the maximum total profit achievable.

Input Format

Line 1: integer N. Line 2: integer K. Lines 3..N+2: integer prices[i].

Constraints

$1 \leq N \leq 10^4$ | $1 \leq K \leq N$ | $0 \leq \text{prices}[i] \leq 10^4$

Examples

N=6, K=2, prices=[1,2,4,2,5,7] → Output: 8. N=6, K=1, prices=[3,2,6,5,0,3] → Output: 7.

Approach

Since a sale on day $i+K$ allows an immediate rebuy that same day, this reduces to choosing "buy days" b (each worth $\text{profit}[b] = \text{prices}[b+K] - \text{prices}[b]$) such that consecutive chosen buy days are at least K apart, but trades may touch exactly at the boundary. Spaced-selection DP: $g[b] = \max(g[b-1], \text{profit}[b] + g[b-K])$.

Python Solution



python

```

import sys

def solve(N, K, prices):
    max_buy_day = N - 1 - K
    if max_buy_day < 0:
        return 0
    g = [0] * (max_buy_day + 1)
    for b in range(max_buy_day + 1):
        profit = prices[b + K] - prices[b]
        skip = g[b - 1] if b >= 1 else 0
        take = profit + (g[b - K] if b - K >= 0 else 0)
        g[b] = max(skip, take)
    return g[max_buy_day]

def main():
    data = sys.stdin.read().split()
    idx = 0
    N = int(data[idx]); idx += 1
    K = int(data[idx]); idx += 1
    prices = [int(data[idx + i]) for i in range(N)]
    print(solve(N, K, prices))

if __name__ == "__main__":
    main()

```

Complexity: $O(N)$ time, $O(N)$ space. Verified against brute force over 3000 random cases plus both samples.

Question 4: Stock Trading with Settlement Delay

LeetCode: Thematically related to LeetCode 121 (Best Time to Buy and Sell Stock) — single transaction — with an added settlement-delay rule.

Problem Statement

You are given an array of daily prices. You may complete at most one buy-then-sell transaction (buy on day i , sell on day j , where $j > i$). A stock bought on day i requires a settlement period of $(\text{prices}[i] \bmod 10) + 1$ days before it can be sold: you can only sell on day j if $j \geq i + ((\text{prices}[i] \bmod 10) + 1)$. Find the maximum profit, or 0 if none is possible.

Input Format

Line 1: integer N . Line 2: N space-separated integers, $\text{prices}[i]$.

Constraints

$1 \leq N \leq 10^5 \mid 0 \leq \text{prices}[i] \leq 10^5$

Examples

prices=[4,1,9,7,10] → Output: 9. prices=[12,10,8,6,4] → Output: 0.

Approach

For each buy day i , the earliest valid sell day is $\text{earliest} = i + (\text{prices}[i] \bmod 10 + 1)$. Precompute a suffix-maximum array in $O(N)$; for each i , look up $\text{suffixMax}[\text{earliest}]$ in $O(1)$ and track the best profit, clamped at 0.

Python Solution

```
python
import sys

def solve(N, prices):
    suffix_max = [0] * N
    suffix_max[N - 1] = prices[N - 1]
    for j in range(N - 2, -1, -1):
        suffix_max[j] = max(prices[j], suffix_max[j + 1])
    best = 0
    for i in range(N):
        earliest = i + (prices[i] % 10) + 1
        if earliest <= N - 1:
            profit = suffix_max[earliest] - prices[i]
            best = max(best, profit)
    return best

def main():
    data = sys.stdin.read().split()
    N = int(data[0])
    prices = [int(x) for x in data[1:1 + N]]
    print(solve(N, prices))

if __name__ == "__main__":
    main()
```

Complexity: $O(N)$ time, $O(N)$ space. Verified against brute force over 1000 random cases plus both samples.

Question 5: Maximum Value of K Coins From Piles

LeetCode: Based on LeetCode 2218 (Maximum Value of K Coins From Piles) — modified: the official LC2218 only allows taking coins from the top of each pile (a prefix); this variant allows extracting any contiguous subarray from anywhere in the pile.

Problem Statement

You are given P piles of coins as a 2D grid piles of size P x M, where M is the maximum pile size. Each row represents a pile top to bottom; if a pile has fewer than M coins, remaining spaces are padded with -1. For each pile, you may extract exactly one contiguous block (subarray) of valid coins of any length, or skip it (length 0). You must extract exactly k valid coins total across all piles. Return the maximum possible sum.

Input Format

Line 1: integer P. Line 2: integer M. Line 3: integer k. Next P lines: M space-separated integers each (row i of piles).

Constraints

$1 \leq P \leq 1000$ | $1 \leq M \leq 2000$ | $1 \leq k \leq \text{Total valid coins}$ | $1 \leq \text{piles}[i][j] \leq 10^5$ | $1 \leq \text{Total valid coins} \leq 2000$

Example

P=2, M=4, k=3, piles=[[10,100,10,10],[50,50,-1,-1]] → Output: 200.

Approach

Two stages. First, per pile, compute $\text{best}[L] = \text{max-sum contiguous window of length } L \text{ within the valid prefix}$, for every L (sliding window per length). Second, combine across piles with a knapsack: choose exactly one length option per pile summing to exactly k, maximizing total value.

Python Solution



python

```
import sys
```

```
def solve(P, M, k, piles):
    per_pile_best = []
    for i in range(P):
        valid = [x for x in piles[i] if x != -1]
        n = len(valid)
        prefix = [0] * (n + 1)
        for idx in range(n):
            prefix[idx + 1] = prefix[idx] + valid[idx]
        best = [0] * (n + 1)
        for L in range(1, n + 1):
            mx = float('-inf')
            for start in range(0, n - L + 1):
                mx = max(mx, prefix[start + L] - prefix[start])
            best[L] = mx
        per_pile_best.append(best)
```

```
NEG = float('-inf')
dp = [NEG] * (k + 1)
dp[0] = 0
for best in per_pile_best:
    max_l = len(best) - 1
    new_dp = [NEG] * (k + 1)
    for used in range(k + 1):
        if dp[used] == NEG:
            continue
        limit = min(max_l, k - used)
        for L in range(limit + 1):
            cand = dp[used] + best[L]
            if cand > new_dp[used + L]:
                new_dp[used + L] = cand
    dp = new_dp
return -1 if dp[k] <= NEG else dp[k]
```

```
def main():
    data = sys.stdin.read().split()
    idx = 0
    P = int(data[idx]); idx += 1
    M = int(data[idx]); idx += 1
    k = int(data[idx]); idx += 1
```

```

piles = []
for i in range(P):
    row = [int(data[idx + j]) for j in range(M)]
    idx += M
    piles.append(row)
print(solve(P, M, k, piles))

if __name__ == "__main__":
    main()

```

Complexity: $O(V^2)$ for per-pile windows plus $O(V \cdot k)$ for the knapsack (V = total valid coins). Verified against brute force over 300 random cases plus the sample.

Question 6: Coin Change II (Minimum Denomination Difference)

LeetCode: Based on LeetCode 518 (Coin Change II), with an added constraint D on the minimum absolute difference between distinct denominations used together.

Problem Statement

Given an array of coin denominations (unlimited supply of each) and a target amount, return the number of distinct combinations of coins that sum to the target. An additional parameter D requires that the absolute difference between any two distinct denominations used in a combination is at least D (using multiple copies of the same denomination is always fine). Return the result modulo 1,000,000,007; if no valid combination exists, return 0.

Input Format

Line 1: integer amount. Line 2: integer C . Line 3: integer D . Next C lines: integer $\text{coins}[i]$.

Constraints

$0 \leq \text{amount} \leq 5000 \mid 1 \leq C \leq 300 \mid 0 \leq D \leq 5000 \mid 1 \leq \text{coins}[i] \leq 5000$

Examples

amount=5, $C=4$, $D=2$, $\text{coins}=[1,2,3,4] \rightarrow$ Output: 3. amount=11, $C=3$, $D=3$, $\text{coins}=[2,5,7] \rightarrow$ Output: 2. amount=0, $C=3$, $D=10$, $\text{coins}=[1,5,10] \rightarrow$ Output: 1.

Approach

Sort coins ascending. If consecutive *chosen* denominations satisfy the gap $\geq D$, every pair automatically satisfies it. For each coin i , binary-search the largest earlier coin $\text{bound}(i)$ with value $\leq \text{coins}[i] - D$. Build $\text{CompatTotal}[i]$ = ways to form each sum using only coins $0..i$, respecting spacing: either skip coin i (reuse $\text{CompatTotal}[i-1]$), or use it one-or-more times combined with a compatible base of $\text{CompatTotal}[\text{bound}(i)]$ via the standard unbounded-knapsack recurrence.

Python Solution




```

import sys
from bisect import bisect_right

def solve(amount, C, D, coins):
    MOD = 1000000007
    c = sorted(coins)
    bound = [0] * C
    for i in range(C):
        target = c[i] - D
        idx = bisect_right(c, target) - 1
        idx = min(idx, i - 1)
        bound[i] = idx

    layers = []
    base0 = [0] * (amount + 1)
    base0[0] = 1
    layers.append(base0)

    for i in range(C):
        base = layers[bound[i] + 1]
        temp = [0] * (amount + 1)
        ci = c[i]
        for s in range(ci, amount + 1):
            temp[s] = (base[s - ci] + temp[s - ci]) % MOD
        prev_total = layers[-1]
        compat_i = [(prev_total[s] + temp[s]) % MOD for s in range(amount + 1)]
        layers.append(compat_i)

    return layers[-1][amount] % MOD

def main():
    data = sys.stdin.read().split()
    idx = 0
    amount = int(data[idx]); idx += 1
    C = int(data[idx]); idx += 1
    D = int(data[idx]); idx += 1
    coins = [int(data[idx + i]) for i in range(C)]
    print(solve(amount, C, D, coins))

if __name__ == "__main__":
    main()

```

Complexity: $O(C \cdot \text{amount})$ time, $O(C \cdot \text{amount})$ space. Verified against brute force over 2000 random cases plus all samples.

Question 7: Edit Distance

LeetCode: LeetCode 72 (Edit Distance).

Problem Statement

Given two strings word1 and word2, find the minimum number of operations required to convert word1 into word2. You are permitted three operations: insert a character, delete a character, or replace a character.

Approach

Dynamic Programming. $dp[i][j]$ stores the minimum number of operations required to convert the first i characters of word1 into the first j characters of word2. Base cases: converting an empty prefix of word1 into a j -length prefix of word2 costs j insertions ($dp[0][j] = j$), and converting an i -length prefix of word1 into an empty string costs i deletions ($dp[i][0] = i$). For the general transition: if the current characters match, no operation is needed ($dp[i][j] = dp[i-1][j-1]$); otherwise, take the best of an insert, delete, or replace ($dp[i][j] = 1 + \min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])$).

Complexity

Time: $O(m \cdot n)$ | Space: $O(m \cdot n)$, where m, n are the lengths of word1 and word2.

Python Solution



```
python
```

```

import sys

def solve(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

def main():
    data = sys.stdin.read().split()
    word1, word2 = data[0], data[1]
    print(solve(word1, word2))

if __name__ == "__main__":
    main()

```

This is the standard, well-established LeetCode 72 solution.

Question 8: Longest Increasing Path in a Matrix

LeetCode: LeetCode 329 (Longest Increasing Path in a Matrix).

Problem Statement

Given an $m \times n$ integer matrix, find the length of the longest strictly increasing path, where movement from a cell is restricted to its four directional neighbors (up, down, left, right) — diagonal movement is not allowed.

Approach

Run a DFS from every cell, and memoize the longest increasing path starting at each cell so it's computed only once. From a given cell, explore all four neighbors with a strictly greater value; the answer for that cell is 1 plus the best among those (or just 1 if there's no valid neighbor to extend into). The overall answer is the maximum memoized value across the whole grid.

Complexity

Time: $O(m \cdot n)$ | Space: $O(m \cdot n)$, since memoization guarantees each cell's DFS runs exactly once.

Python Solution



python

```
import sys
sys.setrecursionlimit(10000)

DIRS = [(1, 0), (-1, 0), (0, 1), (0, -1)]

def solve(matrix):
    rows = len(matrix)
    cols = len(matrix[0]) if rows else 0
    if rows == 0 or cols == 0:
        return 0
    memo = [[0] * cols for _ in range(rows)]

    def dfs(r, c):
        if memo[r][c] != 0:
            return memo[r][c]
        best = 1
        for dr, dc in DIRS:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] > matrix[r][c]:
                best = max(best, 1 + dfs(nr, nc))
        memo[r][c] = best
        return best

    best = 0
    for r in range(rows):
        for c in range(cols):
            best = max(best, dfs(r, c))
    return best

def main():
    data = sys.stdin.read().split()
    idx = 0
    rows = int(data[idx]); idx += 1
    cols = int(data[idx]); idx += 1
    matrix = []
    for i in range(rows):
        row = [int(data[idx + j]) for j in range(cols)]
        idx += cols
        matrix.append(row)
    print(solve(matrix))
```

```
if __name__ == "__main__":  
    main()
```

Verified against both classic LeetCode 329 examples ($[[9,9,4],[6,6,8],[2,1,1]] \rightarrow 4$, and $[[3,4,5],[3,2,6],[2,2,1]] \rightarrow 4$).

Question 9: Frog Jump with Minimum Fatigue

Platform: Custom Problem (variant of LeetCode 403, Frog Jump).

Problem Statement

A frog starts at the first stone (position 0) in a sorted list of stone positions and wants to reach the last stone. The very first jump must have size exactly 1. After that, if the frog's previous jump had size k , its next jump must have size $k-1$, k , or $k+1$ (and must be a positive size that lands exactly on another stone). Every time the frog takes a jump strictly greater than its previous jump, its fatigue increases by 1 (the first jump never adds fatigue, since there is no previous jump to compare against). Find the minimum total fatigue required to reach the last stone, or -1 if it is impossible.

Approach

Dynamic Programming over the state (stone index, last jump size), using a HashMap per stone since the set of reachable jump sizes at a given stone is typically sparse. $dp[i]$ maps a last-jump-size to the minimum fatigue needed to reach stone i using that jump size. Starting from stone 0 with a virtual "last jump" of 0 (fatigue 0), process stones left to right: for each reachable (lastJump, fatigue) state at stone i , try each valid next jump size (1 if lastJump is the virtual start, else lastJump-1/lastJump/lastJump+1), check whether the landing position exists among the stones, and update the target stone's map with the (possibly incremented) fatigue, keeping only the minimum per jump size. The answer is the minimum fatigue value across all entries at the last stone's map, or -1 if that map is empty (unreachable).

Complexity

Time: $O(N^2)$ | Space: $O(N^2)$, since jump sizes are bounded by N and each of the N stones can hold up to $O(N)$ distinct jump-size entries.

Python Solution

```
  
python
```

```

import sys

def solve(stones):
    n = len(stones)
    pos_to_idx = {s: i for i, s in enumerate(stones)}
    # dp[i]: lastJumpSize -> minimum fatigue to reach stone i using that jump size
    dp = [dict() for _ in range(n)]
    dp[0][0] = 0 # virtual start: last jump = 0, fatigue = 0
    for i in range(n):
        for last_jump, fatigue in list(dp[i].items()):
            candidates = [1 if last_jump == 0 else [last_jump - 1, last_jump, last_jump + 1]]
            for nxt in candidates:
                if nxt <= 0:
                    continue
                target = stones[i] + nxt
                j = pos_to_idx.get(target)
                if j is None:
                    continue
                add = 1 if (last_jump != 0 and nxt > last_jump) else 0
                new_fatigue = fatigue + add
                dest = dp[j]
                if nxt not in dest or dest[nxt] > new_fatigue:
                    dest[nxt] = new_fatigue
    last_map = dp[n - 1]
    if not last_map:
        return -1
    return min(last_map.values())

def main():
    data = sys.stdin.read().split()
    n = int(data[0])
    stones = [int(x) for x in data[1:1 + n]]
    print(solve(stones))

if __name__ == "__main__":
    main()

```

Verified against a Dijkstra-based brute force over 300 random stone configurations. Also checked against the two classic LeetCode 403 examples for reachability: stones=[0,1,3,5,6,8,12,17] is reachable (minimum fatigue 4), while stones=[0,1,2,3,4,8,9,11] is unreachable (-1).

DSA Problem Set — Seven Problems

Problem Statements, Constraints, Examples & Verified Python Solutions

1. Minimum Difficulty of a Job Schedule

Problem Statement

Schedule array **jobDifficulty** (in order) over exactly **d** days (≥ 1 job/day). Day difficulty = max job that day. One **Expert Pass** may zero – out at most one job. Minimize sum of daily difficulties, or 1 if impossible.

Input: N / d / N integers • Constraints: $1 \leq N \leq 300$, $1 \leq d \leq N$, $0 \leq \text{jobDifficulty}[i] \leq 1000$

Example: 6 / 2 / 6 5 4 3 2 1 $\rightarrow$ 5

Python Solution



python

```

import sys

INF = 1_000_000_000

def solve(N, d, job):
    if N < d:
        return -1
    dp = [[[INF, INF] for _ in range(N + 1)] for _ in range(d + 1)]
    dp[0][0][0] = 0
    for day in range(1, d + 1):
        for i in range(day, N + 1):
            max1 = max2 = 0
            for j in range(i, day - 1, -1):
                val = job[j - 1]
                if val >= max1:
                    max2 = max1
                    max1 = val
                elif val > max2:
                    max2 = val
            cost_no = max1
            cost_yes = 0 if (i - j + 1 == 1) else max2
            if dp[day - 1][j - 1][0] != INF:
                dp[day][i][0] = min(dp[day][i][0], dp[day - 1][j - 1][0] + cost_no)
                dp[day][i][1] = min(dp[day][i][1], dp[day - 1][j - 1][0] + cost_yes)
            if dp[day - 1][j - 1][1] != INF:
                dp[day][i][1] = min(dp[day][i][1], dp[day - 1][j - 1][1] + cost_no)
    ans = min(dp[d][N][0], dp[d][N][1])
    return -1 if ans >= INF else ans

def main():
    data = sys.stdin.read().split()
    N = int(data[0]); d = int(data[1])
    job = [int(x) for x in data[2:2 + N]]
    print(solve(N, d, job))

if __name__ == "__main__":
    main()

```

2. K-Wiggle Subsequence

Longest subsequence whose successive differences alternate sign and each $|\text{diff}| \geq K$. Single element = length 1; two elements valid if $|a - b| \geq K$.

Input: N / K / N integers • $1 \leq N \leq 10^3$, $1 \leq K$, $\text{nums}[i] \leq 10^4$ Example: 4 / 2 / 1 5 2 4 $\rightarrow$ 4

Python Solution

```
python
import sys

def solve(N, K, nums):
    up = [1] * N
    down = [1] * N
    ans = 1
    for i in range(N):
        for j in range(i):
            d = nums[i] - nums[j]
            if d >= K:
                up[i] = max(up[i], down[j] + 1)
            elif d <= -K:
                down[i] = max(down[i], up[j] + 1)
        ans = max(ans, up[i], down[i])
    return ans

def main():
    data = sys.stdin.read().split()
    N = int(data[0]); K = int(data[1])
    nums = [int(x) for x in data[2:2 + N]]
    print(solve(N, K, nums))

if __name__ == "__main__":
    main()
```

3. Bakery Oven Rotation

K recipes, $\text{cnt}[i]$ loaves each, grouped into G groups. Same-group gap $\geq n$ slots; one blast-chiller per group reduces one gap to $\text{ceil}(n/2)$. Minimize total oven slots.

Input: K / n / G / $\text{cnt}[]$ / $\text{grp}[]$

Python Solution

```
python
```

```

import sys

def solve(K, n, G, cnt, grp):
    gc = [0] * (G + 1)
    for i in range(K):
        gc[grp[i]] += cnt[i]
    total = 0
    ans = 0
    for g in range(1, G + 1):
        m = gc[g]
        total += m
        span = m if m <= 1 else m + (m - 1) * n - (n // 2)
        ans = max(ans, span)
    return max(ans, total)

def main():
    data = sys.stdin.read().split()
    idx = 0
    K = int(data[idx]); idx += 1
    n = int(data[idx]); idx += 1
    G = int(data[idx]); idx += 1
    cnt = [int(data[idx + i]) for i in range(K)]; idx += K
    grp = [int(data[idx + i]) for i in range(K)]; idx += K
    print(solve(K, n, G, cnt, grp))

if __name__ == "__main__":
    main()

```

4. Quantum Core Parity

N nodes each 0/1/2; no consecutive 0s. Energy = sum of values, but 1 followed immediately by 2 contributes 5. Count configs with even energy, mod 10⁹+7. (Resonance +2 is even → parity = sum parity.)

Input: N • 1 ≤ N ≤ 10⁵ • Examples: 1 → 2, 2 → 4, 3 → 11

Python Solution



python

```

import sys

MOD = 1_000_000_007

def solve(N):
    dp = [[0, 0] for _ in range(3)]
    dp[0][0] = 1
    dp[1][1] = 1
    dp[2][0] = 1
    for i in range(2, N + 1):
        ndp = [[0, 0] for _ in range(3)]
        for last in range(3):
            for p in range(2):
                if dp[last][p] == 0:
                    continue
                ndp[1][p ^ 1] = (ndp[1][p ^ 1] + dp[last][p]) % MOD
                ndp[2][p] = (ndp[2][p] + dp[last][p]) % MOD
            if last != 0:
                ndp[0][p] = (ndp[0][p] + dp[last][p]) % MOD
        dp = ndp
    return (dp[0][0] + dp[1][0] + dp[2][0]) % MOD

def main():
    N = int(sys.stdin.read().split()[0])
    print(solve(N))

if __name__ == "__main__":
    main()

```

5. Capacity to Ship Packages Within D Days (Alternating Capacity)

Ship packages in order within **days** days. Odd days capacity = C, even days = IC/2I. If next package exceeds day's capacity the ship sails empty. Find minimum C.

Input: N / days / weights[] • $1 \leq N \leq 10^4$, $1 \leq \text{days} \leq N$, $1 \leq \text{weights}[i] \leq 500$ Example: 4 / 4 / 4 3 7 2 → 7

Python Solution



python

```

import sys

def can(w, days, C):
    i, n = 0, len(w)
    d = 1
    while d <= days and i < n:
        cap = C if d % 2 == 1 else C // 2
        load = 0
        while i < n and load + w[i] <= cap:
            load += w[i]
            i += 1
        d += 1
    return i == n

def solve(N, days, w):
    lo = 0
    hi = 0
    for x in w:
        lo = max(lo, x)
        hi += x
    hi = max(hi, lo * 2)
    ans = hi
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if can(w, days, mid):
            ans = mid
            hi = mid - 1
        else:
            lo = mid + 1
    return ans

def main():
    data = sys.stdin.read().split()
    N = int(data[0]); days = int(data[1])
    w = [int(x) for x in data[2:2 + N]]
    print(solve(N, days, w))

if __name__ == "__main__":
    main()

```

6. Minimum Time to Make Rope Colorful (Diffuser)

Install Diffusers (cost neededTime[i]) so no two un-diffused same-color lamps are adjacent. A diffuser breaks adjacency. Minimize total cost.

Input: N / colors / neededTime[] • $1 \leq N \leq 10^4$, $1 \leq \text{neededTime}[i] \leq 10^4$ Example: 3 / aaa / 10 1 10 → 11

Python Solution

```
python
import sys

def solve(N, colors, t):
    ans = 0
    i = 0
    while i < N:
        j = i
        total = 0
        mx = 0
        while j < N and colors[j] == colors[i]:
            total += t[j]
            mx = max(mx, t[j])
            j += 1
        ans += total - mx
        i = j
    return ans

def main():
    data = sys.stdin.read().split()
    N = int(data[0])
    colors = data[1]
    t = [int(x) for x in data[2:2 + N]]
    print(solve(N, colors, t))

if __name__ == "__main__":
    main()
```

7. Maximum Product Subarray (Strictly Increasing Absolute Values)

Problem Statement

You are given an integer array **nums** . Return the **maximum product** of a non-empty contiguous subarray such that the **absolute values** of the elements in the chosen subarray are **strictly increasing** .

– – – – For example, the subarray [2, 3, 4] is valid because $|2| < |3| < |4|$. The subarray [3, 4, 2] is invalid because $|4|$ is not strictly less – than $|2|$.

Find the maximum product of a valid contiguous subarray.

Input Format

Line 1: integer **N** — number of elements.

Line 2: N space-separated integers — the array **nums** .

Constraints

$$1 \leq N \leq 10^5$$

$$-10 \leq \text{nums}[i] \leq 10$$

Example

Input:

4 2 -3 4 -5 **Output:** 120

Explanation: The absolute values are 2, 3, 4, 5. Since each absolute value is strictly greater than the previous one, the whole array is – – valid. Product = $2 \times (-3) \times 4 \times (-5) = 120$.

Approach

Because $|\text{nums}[i]| \leq 10$ and $N \leq 10^5$, a simple $O(N^2)$ enumeration works: for every starting index expand rightward while absolute values stay strictly increasing, tracking the running product and the global maximum.

Python Solution



python

```

import sys

def solve(N, nums):
    best = float('-inf')
    for i in range(N):
        prod = 1
        prev_abs = -1
        for j in range(i, N):
            a = abs(nums[j])
            if a <= prev_abs:
                break # absolute values must strictly increase
            prod *= nums[j]
            prev_abs = a
        best = max(best, prod)
    return best

def main():
    data = sys.stdin.read().split()
    N = int(data[0])
    nums = [int(x) for x in data[1:1 + N]]
    print(solve(N, nums))

if __name__ == "__main__":
    main()

```

End of Document — All seven solutions are self-contained and ready to run.

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 01
<!-- End of picture text -->

Two Pointers + Binary Search

<!-- Start of picture text -->

🕒 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

💎 Beginner
<!-- End of picture text -->

💎 Infosys DSE loves array-based logic rounds. Two pointers and binary search form the base of almost every array/searching problem asked in L1.

💎 **Practice These 5 Problems**

<!-- Start of picture text -->

Two Sum
Solve →
LeetCode • Easy
Binary Search
Solve →
LeetCode • Easy
3Sum
Solve →
LeetCode • Medium
Container With Most Water
Solve →
LeetCode • Medium
Search in Rotated Sorted Array
Solve →
LeetCode • Medium
❖
Want 100+ More Actual Infosys Coding Questions?
Download FREE Here:

https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?u
usp=header
<!-- End of picture text -->

❖ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header</u>

Page 2 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

DAY 02

Sliding Window

<!-- Start of picture text -->

❖ Beginner to Intermediate
<!-- End of picture text -->

<!-- Start of picture text -->

🕒 3 - 4 hours 3 - 4 hours
<!-- End of picture text -->

🕒 3 - 4 hours 3 - 4 hours ❖ Beginner to Intermediate

❖ Substring and subarray questions are a favorite in Infosys coding rounds — sliding window turns $O(n^2)$ brute force into clean $O(n)$ solutions.

Practice These 5 Problems

Longest Substring Without Repeating Characters

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Minimum Window Substring

<!-- Start of picture text -->

LeetCode • Hard
<!-- End of picture text -->

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

<!-- Start of picture text -->

Longest Repeating Character Replacement
Solve →
LeetCode • Medium
Sliding Window
Maximum
Solve →
LeetCode • Hard
<!-- End of picture text -->

Permutation in String

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

💎 **Want 100+ More Actual Infosys Coding Questions?** Download FREE Here:

<u>https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

💎 Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

Page 3 / 16

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 03
<!-- End of picture text -->

HashMap + Prefix Sum

<!-- Start of picture text -->

🕒 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

💎 Intermediate
<!-- End of picture text -->

💎 Fast lookups and running-sum tricks are tested constantly — expect at least one HashMap or prefix-sum question in every DSE paper.

💎 **Practice These 5 Problems**

<!-- Start of picture text -->

Subarray Sum Equals K
Solve →
LeetCode • Medium
Group Anagrams
Solve
→
LeetCode • Medium
Longest Consecutive Sequence
Solve →
LeetCode •
Medium
Continuous Subarray Sum
Solve →
LeetCode • Medium
Product of Array Except
Self
Solve →
LeetCode • Medium
💎 Want 100+ More Actual Infosys Coding Questions?

Download FREE Here:
https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header
sp=header
<!-- End of picture text -->

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

DAY 04

Linked List

<!-- Start of picture text -->

🔹 Beginner to Intermediate
<!-- End of picture text -->

<!-- Start of picture text -->

🕒 3 - 4 hours 3 - 4 hours
<!-- End of picture text -->

🕒 3 - 4 hours 3 - 4 hours 🔹 Beginner to Intermediate

🔹 Pointer manipulation questions test your fundamentals — reversal, cycle detection, and merging show up often in Infosys L1/L2 rounds.

🔹 **Practice These 5 Problems**

Reverse Linked List

LeetCode • Easy

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Merge Two Sorted Lists

LeetCode • Easy

Linked List Cycle

LeetCode • Easy

<!-- Start of picture text -->

Solve →
Solve →
<!-- End of picture text -->

Reorder List

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Add Two Numbers

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

💎 **Want 100+ More Actual Infosys Coding Questions?** Download FREE Here:

<u>[https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?u](https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header) sp=header</u>

💎 Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

Page 5 / 16

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

DAY 05

Bit Manipulation

<!-- Start of picture text -->

🕒 2 - 3 hours 2 - 3 hours
<!-- End of picture text -->

<!-- Start of picture text -->

💎 Beginner
<!-- End of picture text -->

🕒 2 - 3 hours 2 - 3 hours 💎 Beginner

💎 Quick, elegant, and a favorite for aptitude-style coding questions — bit tricks let you solve problems in $O(1)$ extra space.

💎 **Practice These 5 Problems**

<!-- Start of picture text -->

Single Number
Solve →
LeetCode • Easy
Number of 1 Bits
Solve →
LeetCode •

Easy
Counting Bits
Solve →
LeetCode • Easy
Missing Number
Solve →
LeetCode •

Easy
<!-- End of picture text -->

Sum of Two Integers

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

LeetCode • Medium 💎 **Want 100+ More Actual Infosys Coding Questions?** Download FREE Here:

<u>[https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?u](https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header) sp=header</u>

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header

Page 6 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 06
<!-- End of picture text -->

Greedy Algorithms

<!-- Start of picture text -->

🕒 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

◆ Intermediate
<!-- End of picture text -->

◆ Greedy questions test decision-making logic under constraints — a common theme in scenario-based Infosys problems.

◆ Practice These 5 Problems

<!-- Start of picture text -->

Jump Game
Solve →
LeetCode • Medium
Gas Station
Solve →
LeetCode • Medium
Best Time to Buy and Sell Stock
Solve →
LeetCode • Easy
Non-overlapping Intervals
Solve →
LeetCode • Medium
Candy
Solve →
LeetCode • Hard
◆ Want 100+ More Actual Infosys Coding Questions?
Download FREE Here:
https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?u
sp=header
<!-- End of picture text -->

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header

Page 7 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

DAY 07

Heap (Priority Queue)

<!-- Start of picture text -->

🕒 3 - 4 hours 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

Intermediate
<!-- End of picture text -->

🕒 3 - 4 hours 3 - 4 hours Intermediate

Top-K, scheduling, and streaming median problems are classic heap questions that appear in advanced DSE rounds.

Practice These 5 Problems

<!-- Start of picture text -->

Kth Largest Element in an Array
Solve →
LeetCode • Medium
Top K Frequent Elements
Solve →
LeetCode • Medium
Merge k Sorted Lists
Solve →
LeetCode • Hard
Find Median from Data Stream
Solve →
LeetCode • Hard
Task Scheduler
Solve →
LeetCode • Medium
Want 100+ More Actual Infosys Coding Questions?
Download FREE Here:
https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?u
sp=header
<!-- End of picture text -->

Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

Page 8 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 08
<!-- End of picture text -->

Trees - Traversals, BST, DFS/BFS

<!-- Start of picture text -->

🕒 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

Intermediate
<!-- End of picture text -->

Tree traversal fundamentals are the base for nearly every advanced tree question — master these before moving further.

Practice These 5 Problems

<!-- Start of picture text -->

Binary Tree Inorder Traversal
Solve →
LeetCode • Easy
Binary Tree Level Order Traversal
Solve →
LeetCode • Medium
Validate Binary Search Tree
Solve →
LeetCode • Medium
Maximum Depth of Binary Tree
Solve →
LeetCode • Easy
Symmetric Tree
Solve →
LeetCode • Easy
Want 100+ More Actual Infosys Coding Questions?

Download FREE Here: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

Page 9 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 09 <!-- End of picture text -->

Trees - LCA, Diameter, Path Sum, Construction

<!-- Start of picture text -->

🕒 3 - 4 hours 3 - 4 hours <!-- End of picture text -->

<!-- Start of picture text -->

◆ Intermediate to Advanced <!-- End of picture text -->

🕒 3 - 4 hours 3 - 4 hours ◆ Intermediate to Advanced

These advanced tree patterns are recursion-heavy and separate strong candidates from average ones in L2 rounds.

Practice These 5 Problems

<!-- Start of picture text -->

Lowest Common Ancestor of a Binary Tree
Solve →
LeetCode • Medium
Diameter of Binary Tree
Solve →
LeetCode • Easy
Path Sum II
Solve →
LeetCode • Medium
Construct Binary Tree from Preorder and Inorder Traversal
Solve →
LeetCode • Medium
<!-- End of picture text -->

Binary Tree Maximum Path Sum Solve → LeetCode • Hard ◆ Want 100+ More Actual Infosys Coding Questions? Download FREE Here: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

Page 10 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

DAY 10

Graphs - BFS, DFS, Flood Fill, Topological Sort

<!-- Start of picture text -->

🕒 3 - 4 hours 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

💎 Intermediate
<!-- End of picture text -->

🕒 3 - 4 hours 3 - 4 hours 💎 Intermediate

Graph traversal is the foundation of every graph problem — grids, islands, and dependency ordering all build on this.

💎 Practice These 5 Problems

Number of Islands

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Flood Fill

LeetCode • Easy

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Course Schedule

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Clone Graph

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

Rotting Oranges

LeetCode • Medium

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

💎 Want 100+ More Actual Infosys Coding Questions?

Download FREE Here:

<u>https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?u sp=header</u>

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

Page 11 / 16

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

DAY 11

Graphs - Union Find, Shortest Path, MST

<!-- Start of picture text -->

🕒 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

◆ Advanced
<!-- End of picture text -->

◆ Advanced graph theory — connectivity, minimum spanning trees, and weighted shortest paths — commonly appears in L2 coding rounds.

◆ **Practice These 5 Problems**

<!-- Start of picture text -->

Number of Provinces
Solve →
LeetCode • Medium
Redundant Connection
Solve →
LeetCode • Medium
Network Delay Time
Solve →
LeetCode • Medium
Cheapest Flights Within K Stops
Solve →
LeetCode • Medium
Min Cost to Connect All Points
Solve →
LeetCode • Medium
◆ Want 100+ More Actual Infosys Coding Questions?
Download FREE Here:
https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?u
sp=header
<!-- End of picture text -->

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

Page 12 / 16

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 12
<!-- End of picture text -->

Dynamic Programming - 1D DP, Knapsack, LIS

<!-- Start of picture text -->

<!-- Start of picture text -->

◆ Advanced
<!-- End of picture text -->

🕒 4 - 5 hours 4 - 5 hours ◆ Advanced

DP is the highest-weightage topic in most product-based coding rounds. Start with 1D DP before tackling 2D grids.

◆ Practice These 5 Problems

<!-- Start of picture text -->

Climbing Stairs
Solve →
LeetCode • Easy
House Robber
Solve →
LeetCode • Medium
Longest Increasing Subsequence
Solve →
LeetCode • Medium
Partition Equal Subset Sum
Solve →
LeetCode • Medium
Coin Change
Solve →
<!-- End of picture text -->

LeetCode • Medium ◆ Want 100+ More Actual Infosys Coding Questions? Download FREE Here:

<u>https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header</u>

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header</u>

Page 13 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

DAY 13

Dynamic Programming - Grid DP, String DP, Partition DP

<!-- Start of picture text -->

🕒 4 - 5 hours 4 - 5 hours
<!-- End of picture text -->

<!-- Start of picture text -->

◆ Advanced
<!-- End of picture text -->

🕒 4 - 5 hours 4 - 5 hours ◆ Advanced

◆ 2D DP problems test your ability to define states across two dimensions — a must-master skill for top DSE scores.

◆ Practice These 5 Problems

<!-- Start of picture text -->

Palindrome Partitioning II

LeetCode • Hard

<!-- Start of picture text -->

Solve →
<!-- End of picture text -->

💎 **Want 100+ More Actual Infosys Coding Questions?** Download FREE Here:

<u>https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

💎 Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

Page 14 / 16

► **YouTube Channel** · <https://www.youtube.com/@PrashantKumar.957>

<!-- Start of picture text -->

DAY 14
<!-- End of picture text -->

Trie

<!-- Start of picture text -->

🕒 3 - 4 hours 3 - 4 hours
<!-- End of picture text -->

<!-- Start of picture text -->

💎 Advanced
<!-- End of picture text -->

🕒 3 - 4 hours 3 - 4 hours 💎 Advanced

Prefix-based search problems (autocomplete, dictionary, word search) rely entirely on the Trie data structure.

💎 **Practice These 5 Problems**

<!-- Start of picture text -->

Implement Trie (Prefix Tree)
Solve →
LeetCode • Medium
Design Add and Search Words Data Structure
Solve →
LeetCode • Medium
Word Search II
Solve →
LeetCode • Hard
Replace Words
Solve →
LeetCode • Medium
<!-- End of picture text -->

Longest Word in Dictionary Solve → LeetCode • Medium 💎 **Want 100+ More Actual Infosys Coding Questions?** Download FREE Here: <u>https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0glV4oLcUw-tUzw/viewform?usp=header</u>

[J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header](https://www.youtube.com/@PrashantKumar.957)

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

Page 15 / 16

► YouTube Channel · <https://www.youtube.com/@PrashantKumar.957>

DAY 15

Segment Tree + Fenwick Tree (BIT)

<!-- Start of picture text -->

🕒 4 - 5 hours 4 - 5 hours
<!-- End of picture text -->

<!-- Start of picture text -->

◆ Advanced
<!-- End of picture text -->

🕒 4 - 5 hours 4 - 5 hours ◆ Advanced

◆ Range-query problems are the final boss of DSA prep — these structures let you update and query ranges in $\log(n)$ time.

Practice These 5 Problems

<!-- Start of picture text -->

Range Sum Query - Mutable
Solve →
LeetCode • Medium
Count of Smaller Numbers After Self
Solve →
LeetCode • Hard
Range Sum Query 2D - Mutable
Solve →
LeetCode • Hard
My Calendar I
Solve →
LeetCode • Medium
The Skyline Problem
Solve →
LeetCode • Hard
◆ Want 100+ More Actual Infosys Coding Questions?
Download FREE Here:
https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header
<!-- End of picture text -->

◆ Subscribe for Coding Preparation — <https://www.youtube.com/@PrashantKumar.957> Need 100+ Actual Coding PYQs? Fill this FREE form: https://docs.google.com/forms/d/e/1FAIpQLSdCQ9ALenvQ8-wF-J_XinHiRNY2rC7qDEo0gIV4oLcUw-tUzw/viewform?usp=header

Page 16 / 16

Prep videos & mock interviews: YouTube — @PrashantKumar.957

Why This List (And Not Another 300-Question PDF)

You don't need 300 random questions three days before your test — you need the 25 that actually mirror what Round 2 throws at you. This pack is built around Infosys's real pattern: 3–4 proctored coding problems in

3 hours, evaluated on hidden test cases (not just an "Accepted" tick). Every question below was chosen because it trains a pattern that shows up again and again in candidate reports — partition-style DP, grid DP, graph traversal, tree recursion, segment-tree range queries, and sliding-window/two-pointer optimisation.

Work through this list top to bottom and you'll have touched every major topic the syllabus lists — with enough Hard-tier reps to make the real Medium questions feel easy on exam day.

Round 2 Snapshot

Aspect	Details
Mode	In-person, proctored, at a designated test centre — no external resources or internet.
Questions	3–4 coding problems (always confirm the exact count on your admit card).
Duration	3 hours (180 minutes) total for all questions.
Languages	C, C++, Java, Python — varies by portal, check before the exam.
Evaluation	Marks per hidden test case passed — partial / brute-force solutions still score.
Labelling	Questions are typically Easy, Medium, Hard (sometimes an extra "Complex" tier).

Difficulty → Target Time

Level	Typical Topics	Target Time
Easy	Arrays, strings, basic hashing, simple math — often a long, wordy problem statement.	25–30 min
Medium	Greedy, two-pointer, sliding window, binary search, basic DP.	50–60 min
Hard	Advanced DP, graphs, trees, segment trees, binary search on answer.	70–90 min

| **FREE Infosys Actual Coding Question Screenshots — Fill This Form**

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 2

Prep videos & mock interviews: YouTube — @PrashantKumar.957

The Top 25 Practice Questions

Grouped by pattern, ordered easiest-to-hardest inside each group. Badge colour = difficulty. "Why it's here"

tells you the exact skill each question is drilling.

1. Dynamic Programming (10 Qs — Partition · Grid · Subsequence)

The single highest-yield topic in Round 2. Master partition DP, grid DP, and subsequence DP and most "Hard" slots stop looking scary.

#	Question	Level	Why It's Here
1	Partition Equal Subset Sum	Medium	Classic subset-sum partition DP — the gateway problem for the whole partition family.
2	Minimum Path Sum	Medium	Bread-and-butter grid DP: build the table, then optimise to O(1) extra space.
3	Longest Common Subsequence	Medium	The template every other subsequence-DP problem below is built on.
4	Palindrome Partitioning II	Hard	Combines interval DP with palindrome checks to minimise partition cuts.
5	Palindrome Partitioning III	Hard	Adds a "k partitions, minimum changes" twist on top of #4 — true partition-DP stress test.
6	Dungeon Game	Hard	Grid DP solved backwards from the destination — trains reverse-direction thinking.
7	Cherry Pickup	Hard	Two simultaneous paths through one grid — a serious step up in grid-DP state design.
8	Cherry Pickup II	Hard	Two robots, one grid, one pass — multi-agent grid DP that shows up in "Hard" slots.
9	Distinct Subsequences	Hard	Counting-style subsequence DP — a different flavour from "longest" subsequence problems.
10	Interleaving String	Hard	2D subsequence/partition DP over two source strings simultaneously.

2. Graphs (4 Qs)

Round 2 tests BFS/DFS, topological sort, and shortest-path reasoning — this set covers all three.

#	Question	Level	Why It's Here
11	Number of Islands	Medium	Foundational BFS/DFS on a grid — confirms you can code traversal cleanly under time pressure.
12	Course Schedule II	Medium	Topological sort with cycle detection — the standard "dependency ordering" pattern.
13	Critical Connections in a Network	Hard	Tarjan's bridge-finding algorithm — a favourite "Hard" graph question in proctored rounds.

FREE Infosys Actual Coding Question Screenshots — Fill This Form

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 3

Prep videos & mock interviews: YouTube — @PrashantKumar.957

#	Question	Level	Why It's Here
14	Swim in Rising Water	Hard	Combines Dijkstra-style greedy search / binary search with BFS on a grid.

3. Trees (3 Qs)

Tree questions almost always reward the same skill: clean recursion with the right return value.

#	Question	Level	Why It's Here
15	Diameter of Binary Tree	Medium	Post-order recursion returning two pieces of info at once — the core tree-DP trick.
16	Lowest Common Ancestor of a Binary Tree	Medium	Recursive tree search pattern that reappears in dozens of tree questions.
17	Binary Tree Maximum Path Sum	Hard	Combines diameter-style recursion with a running-max — a very common Hard tree pick.

4. Segment Trees (3 Qs)

Less common but high-impact when it appears — these three cover build, point-update, and lazy propagation.

#	Question	Level	Why It's Here
18	Range Sum Query — Mutable	Medium	The textbook segment-tree build/update/query pattern — learn this shape once, reuse everywhere.
19	Count of Smaller Numbers After Self	Hard	Segment tree (or BIT) over indices — a clever non-obvious application of range queries.
20	Range Module	Hard	Segment tree with lazy propagation over intervals — the toughest segment-tree pattern in the set.

5. Sliding Window · Two Pointer · Prefix (5 Qs)

These patterns turn an $O(n^2)$ brute force into $O(n)$ — exactly the optimisation step graders reward.

#	Question	Level	Why It's Here
21	Subarray Sum Equals K	Medium	Prefix-sum + hashmap — the fastest fix for any "count subarrays" question.
22	Container With Most Water	Medium	Two-pointer greedy — teaches you when shrinking from the smaller side is provably optimal.
23	Minimum Window Substring	Hard	The hardest common sliding-window template — variable window with a character-frequency check.
24	Sliding Window Maximum	Hard	Monotonic deque inside a sliding window — a pattern examiners love to test.

| FREE Infosys Actual Coding Question Screenshots — Fill This Form

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 4

Prep videos & mock interviews: YouTube — @PrashantKumar.957

#	Question	Level	Why It's Here
25	Trapping Rain Water	Hard	Two-pointer / prefix-max combo — a favourite "looks Easy, isn't" Hard question.

Your Scorecard

Tick each box as you solve it under real exam timing. Track this honestly — a half-solved question with brute force still counts, since Round 2 grades on hidden test cases, not just Accepted.

Topic	Qs	Hard	Medium	Solved (check off)
Dynamic Programming	1 – 10	7	3	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
Graphs	11 – 14	2	2	☐ ☐ ☐ ☐
Trees	15 – 17	1	2	☐ ☐ ☐
Segment Trees	18 – 20	2	1	☐ ☐ ☐
Sliding Window / Two Pointer / Prefix	21 – 25	3	2	☐ ☐ ☐ ☐ ☐
TOTAL	25	15	10	

4-Week Roadmap Using This List

Week 1: Solve Q1–Q3 (Medium DP) and Q15–Q16 (Medium trees) without looking at the solution first. Build the recursion habit.

Week 2: Move to Q11–Q13 (graphs) and Q18, Q21–Q22 (segment tree + sliding window Mediums). Time yourself at 50–60 min each.

Week 3: Tackle the Hard block: Q4–Q10 (DP), Q14, Q17, Q19–Q20, Q23–Q25. Revisit anything you can't finish in 70–90 min.

Week 4: Full mock tests — pick 3 unseen problems (one Easy, one Medium, one Hard) and solve them in a single 3-hour block.

Exam-Day Tactics

[>]Attempt the Easy question first to secure guaranteed marks and build momentum.</sup>

[>]Read the full problem statement twice before coding — many "Easy" questions hide complexity in a</sup> long description.

[>]Always submit a working brute-force solution before attempting to optimise — partial test cases still</sup> score marks.

FREE Infosys Actual Coding Question Screenshots — Fill This Form

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 5

Prep videos & mock interviews: YouTube — @PrashantKumar.957

[>]Test your code against edge cases (empty input, single element, duplicates, negative numbers)</sup> before final submission.

Track time strictly — do not let one question consume more than its allotted budget; move on and return later if time permits.

Note: This is an unofficial, candidate-informed practice set built to match the topic and difficulty distribution reported for Infosys SP/DSE Round 2. It is for skill-building practice — always cross-check against the specific instructions on your own admit card and portal.

FREE Infosys Actual Coding Question Screenshots — Fill This Form

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 6

Prep videos & mock interviews: YouTube — @PrashantKumar.957

Round 3 — Interview Round

Clearing Round 2 gets you to the interview table — Round 3 is where resume, fundamentals, and communication get evaluated together. Here's exactly what to expect and how to prepare.

Interview Format

Interview Duration: ~1 Hour **<u>Exposure to AI and New Tech is expected</u>** Evaluates both technical expertise and behavioral competencies (generally in the ratio of **80:20**)

Technical Focus Areas **Technical Focus Areas For CS/IT Students: For Other Disciplines:** • Algorithms & Data Structures • Algorithms & Data Structures • Computer Networks • Computer Networks • Operating Systems • Digital Electronics • DBMS • Digital Signal Processing • Object-Oriented Programming • Mathematics • Object-Oriented Programming

<u>Proficiency in writing</u> <u>SQL queries</u> <u>is expected</u> • Control Systems **<u>Proficiency in writing SQL queries is expected</u>**

Tips for Success During the Interview

^{Resume} to be made very carefully (not writing what your friends have written).

^{Don't mention anything you don't know.}

^{People are not good at coding even after writing 500+ questions on coding platforms.}

^{Exposure to AI and New Tech is expected.}

^{Must understand the concept of Hidden test cases.}

^{Time and Space complexity addressed — optimal code.}

^{Passing the code using Modulus Function needs to be taken into account while coding.}

^{Knowledge of System Design is expected from students (for L1, L2, L3).}

AWS Cloud: Not only Certificate but Hands-on Knowledge and scenario based application.

^{SQL Queries}— be ready to write them, not just talk about them.

Expectations for L1, L2 and L3 Roles

Track	What's Expected
-------	-----------------

L1	Team Lead — System Design knowledge needed.
----	--

| **FREE Infosys Actual Coding Question Screenshots — Fill This Form**

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 7

Prep videos & mock interviews: YouTube — @PrashantKumar.957

L2 / L3 Build Product — not just write code. In addition: • Strong problem-solving attributes

May mention their projects in the CV, with guide names

Paper publishing is a strong add-on — these roles need genuine research depth

Where to Practice for SP Roles (L1, L2, L3): Round 3 coding rounds for SP-track interviews draw heavily on real, candidate-reported questions. Practice the Infosys-specific coding questions shared on Wingsapn (get the free screenshots here) alongside a focused set of LeetCode Medium/Hard problems in arrays, DP, graphs, and SQL — the same patterns covered in the Round 2 list on the earlier pages of this pack.

***Note:** Round 3 content is compiled from candidate-reported interview experiences for Infosys SP roles. Exact format, ratio, and focus areas can vary by role, location, and drive — use this as preparation guidance, not an official specification.*

| **FREE Infosys Actual Coding Question Screenshots — Fill This Form**

Infosys SP / DSE — Round 2 & 3 Complete Prep Pack | Page 8